Functions and Arrays in C Programming

Simple Teaching Notes for Secondary School Students

Prepared by Vincent Simoni Massawe

Cyber Security and Digital Forensics, Arusha Technical College

How to use this guide: Explain each real-life comparison first, then show the code, then run it and show the output. Encourage students to guess the output before you run each example.

PART 1: Functions in C Programming

1. What Is a Function?

A function is a small, named block of code built to do one specific job. You give it a name, and whenever your program needs that job done, you simply call that name instead of rewriting the same instructions again.

Real-life comparison for your class: A function is like a school bell. The bell has exactly one job — to ring at certain times. It does not sweep the compound or mark registers; it just rings. In the same way, each function in a C program is built to carry out one clear task, and nothing else.

2. Why Do We Use Functions?

- To avoid writing the same code many times (a function can be reused/called as many times as needed).
- To make programs easier to read, since each function has a clear, single purpose.
- To make it easier to find and correct mistakes, because you can test one function at a time.
- To break a big, complicated problem into smaller, manageable pieces.

3. The Basic Structure of a Function

Every function in C follows this general pattern:

```
return_type function_name(parameter_list)
{
    // statements that make up the function body
}
```

return_type is the kind of value the function will give back (for example int, float, or void if it gives nothing back). function_name is what you will use to call it. The parameter_list lists the values the function needs to do its job.

4. Function Declaration vs Function Definition

A declaration simply tells the compiler that a function exists, what it is called, and what type of values it expects and returns — much like an entry in a school register that lists a student's name and class without telling their full story.

A definition is the complete, actual code of the function — the full set of instructions that carry out the task.

```
int findArea(int length, int width);   // declaration only
```

5. Calling a Function

To use a function, you 'call' it by writing its name followed by brackets containing any values it needs. When a function is called, control of the program jumps into that function, runs its instructions, and then returns to where it was called from.

Example 1 — a function that calculates the area of a rectangle

```
#include <stdio.h>

int findArea(int length, int width) {
    int area;
    area = length * width;
    return area;
}

int main() {
    int result;
    result = findArea(6, 4);   // calling the function
    printf("The area is %d\n", result);
    return 0;
}
```

Output

```
The area is 24
```

6. Parameters and Arguments

Parameters are the placeholder names used inside the function's brackets when it is defined (for example, length and width above). Arguments are the actual values given when the function is called (for example, 6 and 4).

7. Return Types and the return Statement

The return statement sends a value back to wherever the function was called, and immediately ends the function. The type of that value must match the function's declared return type (int, float, char, double, and so on).

Example 2 — checking whether a number is even

```
int isEven(int number) {
    if (number % 2 == 0)
        return 1;   // 1 means true: the number is even
    else
        return 0;   // 0 means false: the number is odd
}
```

8. Void Functions

A void function does not return any value. It is used when a function's job is simply to do something (like printing a message), not to calculate and hand back a result.

Example 3 — a void function

```
void showWelcome() {
    printf("Welcome to our Computer Science class!\n");
}

int main() {
```

```
    showWelcome();   // calling a void function
    return 0;
}
```

9. Call by Value vs Call by Reference

Call by value: the function receives only a copy of the value. Any changes made inside the function do not affect the original variable. Comparison: it is like giving someone a photocopy of your homework — if they scribble on it, your original book stays clean.

Call by reference: the function receives the actual memory address of the variable (using a pointer), so changes made inside the function do affect the original value. Comparison: it is like giving someone the actual key to your locker — whatever they do inside, the real locker is changed.

Example 4 — call by value (does NOT change the original)

```
void addBonus(int marks) {
    marks = marks + 5;
    printf("Inside function: %d\n", marks);
}

int main() {
    int score = 60;
    printf("Before: %d\n", score);
    addBonus(score);
    printf("After: %d\n", score);
    return 0;
}
```

Output

```
Before: 60
Inside function: 65
After: 60   // unchanged, because it was a copy
```

Example 5 — call by reference (DOES change the original)

```
void addBonusRef(int *marks) {
    *marks = *marks + 5;
}

int main() {
    int score = 60;
    addBonusRef(&score);   // passing the address of score
    printf("New score: %d\n", score);
    return 0;
}
```

Output

```
New score: 65   // changed, because the function reached the real variable
```

10. Common Mistakes Students Make

- Trying to call a function before it has been declared or defined.
- Forgetting the return statement in a function that is not void.
- Giving the wrong number or type of arguments when calling a function.
- Confusing the assignment operator (=) with the return statement.

11. Class Activity Ideas

- Ask students to write a function that calculates the average of three numbers.
- Ask students to write a void function that prints their name and class/form.
- Split the class into two groups: one writes a call-by-value example, the other call-by-reference — then each group explains their code to the other.

PART 2: Arrays and Strings

1. What Is an Array?

An array is a single variable that can hold many values of the same type, stored in order, one after another.

Real-life comparison: think of a row of numbered lockers in school. Each locker holds one item, and each locker has a number. In C, locker numbering (called the index) starts at 0, not 1 — so the first locker is number 0, the second is number 1, and so on.

2. Declaring a One-Dimensional Array

```
int marks[5];   // an array with room for 5 whole numbers (5 lockers)
```

3. Initialising an Array

```
int marks[5] = {70, 65, 80, 90, 55};
```

4. Accessing and Changing Elements

You reach a value inside an array by writing its index number in square brackets. Remember: counting starts at 0.

Example 6 — storing and updating students' marks

```
#include <stdio.h>

int main() {
    int marks[5] = {70, 65, 80, 90, 55};
    printf("First student's mark: %d\n", marks[0]);
    marks[4] = 60;   // updating the last student's mark
    printf("Updated last mark: %d\n", marks[4]);
    return 0;
}
```

Output

```
First student's mark: 70
Updated last mark: 60
```

5. Two-Dimensional Arrays (Tables/Matrices)

A two-dimensional array is like a table with rows and columns — think of a class timetable, where each row is a day and each column is a lesson period.

Example 7 — marks for 2 students in 2 subjects

```
#include <stdio.h>
```

```
int main() {
    int classMarks[2][2] = {{70, 80}, {60, 90}};
    // classMarks[0][0] = first student's first subject
    printf("Student 1, Subject 1: %d\n", classMarks[0][0]);
    printf("Student 2, Subject 2: %d\n", classMarks[1][1]);
    return 0;
}
```

Output

```
Student 1, Subject 1: 70
Student 2, Subject 2: 90
```

6. Strings in C

A string is simply an array of characters (letters), ending with a special hidden character written \0, which tells the computer where the word stops.

Real-life comparison: a string is like a row of lockers, but each locker holds one letter instead of a whole item — and the very last locker used has a small 'STOP' sign (\0) telling the computer the word has ended.

```
char studentName[10];        // room for up to 9 letters, plus the stop sign
char studentName[] = "Amina";   // declared and given a value directly
```

7. Useful String Functions

- puts() — prints a string to the screen.
- strlen() — tells you how many letters are in a string (not counting the stop sign).
- fgets() — a safer way to read a string typed by the user, because you can limit how many letters it accepts. (Its older cousin, gets(), is best avoided, since it does not limit the length and can overflow.)

Example 8 — printing a name and counting its letters

```
#include <stdio.h>
#include <string.h>

int main() {
    char studentName[] = "Amina";
    puts(studentName);
    int nameLength = strlen(studentName);
    printf("Number of letters: %d\n", nameLength);
    return 0;
}
```

Output

```
Amina
Number of letters: 5
```

8. Common Mistakes Students Make

- Forgetting that array counting starts at index 0, not 1.

- Trying to use an index beyond the array's size — for an array of 5, the last valid index is 4, not 5.
- Declaring a string array too small, forgetting to leave room for the hidden stop character (\0).
- Using gets() in real programs, which is unsafe — fgets() should be used instead.

9. Class Activity Ideas

- Ask students to create an array of 5 favourite numbers and print each one.
- For stronger students: store several classmates' names using an array of strings.
- Group task: use a 2D array to store marks for 3 students in 2 subjects, then print each student's total.

Teaching Tips for You, Vincent

- Always give the real-life comparison first, before showing any code — students grasp a picture or comparison faster than raw syntax.
- Where possible, run the code live (or write the output on the board) so students see the cause-and-effect clearly.
- Keep example numbers small (2–5 values) so students can trace the program by hand on the board.
- Before running any example, ask students to guess the output first — this builds real understanding rather than passive copying.
- Let students change one value in an example and predict what will happen — active experimenting works better than only watching.